

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

Campus Wi-Fi Complaint Management System

*Campus Wi-Fi Complaint Management System — Module 2: Complaint Tracking & Status
Management*

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA0707 – Computer Networks

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

LOKESH S (192572151)

Under the Supervision of

Dr. R. Shankar

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**Campus Wi-Fi Complaint Management System — Module 2: Complaint Tracking & Status Management**” has been carried out by LOKESH S (!92572151) under the supervision of Dr. R. Shankar and is submitted in partial fulfilment of the requirements for the current semester of the B. E Computer Science and Engineering programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR	COURSE FACULTY
Dr. S. SIVAGAMI Professor, Department of Data Vista / CSE, Computer Science and Engineering SIMATS Engineering Chennai – 602105.	Dr. R. Shankar, Designation Department of Computer Science and Engineering SIMATS Engineering, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

DECLARATION

I, LOKESH S (192572151) of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “**Campus Wi-Fi Complaint Management System — Module 2: Complaint Tracking & Status Management**” is the result of my own Bonafide effort. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. I further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. External tools, software, datasets, computational resources (Node.js, Express.js, MySQL, mysql2 driver) and AI-assisted resources used during the project have been appropriately disclosed. I confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 07/09/2026

Signature of the Student with Name

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

Individual Contribution Statement

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. %
LOKESH S (192572151)	Requirement analysis, extension of schema usage, REST endpoint development for tracking/assignment, technician-facing front end, unit & integration testing, documentation	Designed status- transition and assignment logic, built four new Express endpoints, built technician.html and dashboard status view for Module 2	Wrote and executed test cases for status transitions, technician assignment and student status visibility	Authored all chapters of this report	100%

ABSTRACT

Reliable Wi-Fi connectivity is core academic infrastructure, and once a fault is reported, the value of that report depends entirely on whether its progress can be seen and acted upon. Module 1 of the Campus Wi-Fi Complaint Management System gave students a way to register and submit a structured complaint, but the complaint's status column and technician column, though present in the schema, were never populated or exposed after creation — a complaint, once submitted, disappeared from view for both the student and the institution's technical staff. This report documents Module 2 — Complaint Tracking and Status Management — which closes that gap by adding a technician-assignment workflow and a four-state status lifecycle (Pending → Assigned → In Progress → Resolved) on top of the existing users/complaints schema, with no change to the tables designed in Module 1.

Module 2 was implemented as four new REST endpoints on the same Node.js/Express.js/MySQL three-tier architecture used in Module 1: an endpoint that lists all complaints (for administrative and technician visibility), an endpoint that assigns a named technician to a complaint and automatically advances its status to 'Assigned', an endpoint that lets a technician move an assigned complaint through 'In Progress' to 'Resolved', and an extension of the existing student-facing retrieval endpoint so that a student's own complaint list now also returns live status and assigned-technician values. A lightweight technician-facing page (technician.html) was built to consume these endpoints without requiring any change to the student-facing dashboard beyond displaying the two additional fields.

Three alternative approaches to representing and enforcing the status workflow were evaluated — an unconstrained free-text status field, a client-enforced state machine, and a server-enforced state machine validated against a fixed transition table — against weighted criteria of data integrity, implementation effort, and auditability; the server-enforced state machine was selected because it was the only alternative that could guarantee a complaint could not skip or reverse states regardless of what the client sent.

Keywords: Complaint tracking, status transition workflow, technician assignment, Node.js, Express.js, MySQL, REST API.

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Fig. 2.1	Comparative positioning of existing complaint-tracking approaches	—
Fig. 3.1	System architecture — three-tier view with Module 2 scope highlighted	—
Fig. 3.2	Complaint status transition (state) diagram	—
Fig. 3.3	Module 2 process flow: assignment and status update	—
Fig. 4.1	Module 2 milestone timeline (Gantt view)	—

LIST OF TABLES

Table No.	Table Name	Page No.
Table 2.1	Comparative analysis of existing complaint-tracking approaches	—
Table 3.1	Stakeholder / user requirements	—
Table 3.2	Functional and non-functional requirements	—
Table 3.3	Constraints and applicable standards	—
Table 3.4	Design specifications	—
Table 3.5	Alternative-solution evaluation matrix	—
Table 3.6	Trade-off analysis	—
Table 3.7	Sustainability, ethics, safety, and security evaluation	—
Table 3.8	Risk register	—
Table 4.1	Development resources	—
Table 4.2	Tools, software and equipment used	—
Table 4.3	Test plan and verification	—
Table 4.4	Specification vs. achieved values	—
Table 4.5	API response-time results	—
Table 4.6	Objective achievement mapping	—
Table 4.7	Roles and contribution	—
Table 4.8	Cost estimate vs. actual	—

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
API	Application Programming Interface	—
REST	Representational State Transfer	—
HTTP	Hypertext Transfer Protocol	—
JSON	JavaScript Object Notation	—
SQL	Structured Query Language	—
FK	Foreign Key	—
CRUD	Create, Read, Update, Delete	—
CORS	Cross-Origin Resource Sharing	—
UI/UX	User Interface / User Experience	—
SDG	Sustainable Development Goal	—
ms	Milliseconds (API response time)	ms
IT	Information Technology	—

CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Module 1 of the Campus Wi-Fi Complaint Management System gave students at SIMATS Engineering a structured way to register, log in, and submit a Wi-Fi fault complaint, which was stored in a two-table MySQL schema (users, complaints) with a unique, human-readable complaint identifier generated at submission time. That schema already carried a status column (defaulted to 'Pending') and a technician column, but Module 1 deliberately left both unused after insertion: a submitted complaint could be retrieved by the student who filed it, but its status never changed, and no technician was ever recorded against it. In effect, the complaint disappeared from the process the moment it was filed.

This report documents Module 2 — Complaint Tracking and Status Management — the second of the three modules that make up the complete system (Module 1: Complaint Registration & User Management; Module 2: Complaint Tracking & Status Management; Module 3: Admin & Network Issue Management). Module 2 turns the dormant status and technician columns into an active, auditable workflow: complaints can be assigned to a named technician, moved through a defined sequence of states, and the resulting status is made visible to the student who filed the complaint, closing the visibility gap identified as a limitation in the Module 1 report.

1.2 Need for the Project

- Module 1 complaints have no visible progress after submission, so a student who reported a fault has no way to know whether anyone has seen it, is working on it, or has fixed it.
- Network technicians and administrators have no shared queue of outstanding complaints; without an assignment mechanism, two technicians could unknowingly duplicate effort on the same fault, or a complaint could simply be forgotten.
- An un-auditable status field (i.e., one any client could overwrite to any value) would allow a complaint to be marked 'Resolved' without ever having been assigned or worked on, undermining the trustworthiness of the record.

1.3 Problem Statement

Complaints registered through Module 1 currently have a status field that is set once (to 'Pending') and never updated, and a technician field that is never populated, so neither students nor technical staff can see or manage a complaint's progress after submission. The engineering problem is therefore to design and implement a status-tracking and technician-assignment layer that (a) allows an administrator to assign a named technician to any pending complaint, (b) allows that assignment to advance the complaint through a well-defined, forward-only sequence of states culminating in 'Resolved', (c) enforces this sequence on the server so that no client can set an illegal or out-of-order status, (d) exposes the current status and assigned technician back to the student who filed the complaint, and (e) does all of this without modifying the users/complaints schema or any Module 1 endpoint, since Module 1 has already been verified and deployed. This problem inherently involves multiple interacting factors — data integrity for the state machine, usability for both technicians and students, and continued extensibility toward the reporting and prioritisation features planned for Module 3.

1.4 Project Aim

To design, implement and verify a Complaint Tracking and Status Management module (Module 2) that allows an administrator to assign complaints to technicians and allows those complaints to progress through a defined, server-enforced status lifecycle that is visible to the student who filed the original complaint.

1.5 Project Objectives

1. To define a forward-only status lifecycle (Pending → Assigned → In Progress → Resolved) that reuses the existing status column without any schema migration.
2. To implement a technician-assignment endpoint that records a technician name against a complaint and advances its status from Pending to Assigned.
3. To implement a status-update endpoint that validates each requested transition against the defined lifecycle before writing to the database.

1.6 Scope

Included in this report (Module 2):

- A GET /api/complaints endpoint returning every complaint in the system with its current status and assigned technician, for administrative/technician visibility.
- A PUT /api/complaints/:id/assign endpoint that sets the technician field and advances status from 'Pending' to 'Assigned'.
- A PUT /api/complaints/:id/status endpoint that advances an already-assigned complaint from 'Assigned' to 'In Progress' to 'Resolved', rejecting any request that is not the single legal next step.
- Extension of GET /api/complaints/:userId (defined in Module 1) to also return status and technician for each of a student's complaints.
- A technician-facing static page (technician.html) listing complaints assigned to the logged-in technician, with a control to advance status.
- Persistent storage of every assignment and status change in the existing complaints table (no new tables).

Excluded from this report (addressed by Module 3, referenced only as context):

- The administrator dashboard's complaint-volume analytics, prioritisation-by-severity logic, and reporting views.
- Creation, management or authentication of technician accounts as a distinct role (technicians are identified by name string in this module; role-based technician login is a Module 3 concern).
- Email/SMS notification delivery, real-time network monitoring, and AI-based fault prediction (listed as future scope in Chapter 5).

Assumptions and boundary conditions:

- Module 1's users and complaints tables, and its four REST endpoints, remain unchanged and continue to run on the same Express application and MySQL instance.
- Technician names are entered as free text by the administrator at assignment time; a dedicated technicians table is treated as a Module 3 enhancement rather than a Module 2 requirement.
- Only forward transitions are supported; reverting a complaint from 'Resolved' back to an earlier state, or re-assigning a resolved complaint, is out of scope and would require an explicit reopen workflow not specified here.

1.7 Expected Engineering Outcome

A working, testable extension to the existing web application comprising: four Module 2 REST endpoints layered on the unmodified Module 1 schema; a server-enforced status state machine that rejects illegal transitions; and a technician-facing front end through which complaints can be assigned and progressed — verified against the requirements and test plan defined in Chapters 3 and 4, and ready to supply fully-populated status and technician data to the administrative module (Module 3).

CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Existing approaches to complaint/ticket status tracking were reviewed under three headings: (i) generic IT-ticketing and helpdesk platforms and the status models they use; (ii) open-source complaint/grievance-management project templates commonly used in academic capstone work; and (iii) the as-built Module 1 system itself, whose status and technician columns were present but unused prior to this module. The review draws on the reference texts listed in Chapter 6 for the underlying database and workflow concepts, and on inspection of typical ticketing-system status models, to identify the design gap that Module 2 is intended to close.

2.2 Review of Existing Work

Generic enterprise helpdesk and ticketing platforms almost universally implement some form of ticket-status lifecycle (commonly Open/In Progress/Resolved/Closed) and an assignment field, and enforce at least basic rules about which transitions are legal from the current state. This confirms that a small, fixed set of forward-moving states, enforced server-side, is a well-precedented and appropriate design for this class of problem, rather than an ad hoc invention specific to this project.

Open-source complaint-management templates built on LAMP or MEAN/MERN stacks typically represent status as a plain string column with no server-side transition validation, leaving the front end responsible for only ever sending 'sensible' values. This is simple to implement but does not prevent a malformed or malicious request from setting an arbitrary status value directly through the API, which is precisely the risk Module 2 is designed to close by validating transitions on the server rather than trusting the client.

Reviewing the as-built Module 1 system directly showed that the status column already defaults correctly to 'Pending' and the technician column already exists and is nullable, meaning the database schema needed for Module 2 was already complete; the gap was entirely in the missing API and workflow logic, not in the data model.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Generic enterprise ticketing platform (Open/In Progress/Resolved/Closed)	Proven status model; familiar to technicians; often supports SLAs	Not tailored to Wi-Fi fault vocabulary; licensing/hosting cost; heavier than needed for a single-campus deployment	Confirms the value of a small, fixed status set as the target model, adapted here to four Wi-Fi-relevant states
Open-source LAMP/MERN complaint templates (free-text status)	Simple to implement; free and self-hostable	No server-side transition validation; a client can set any status value, risking an inconsistent record	Identifies the specific gap (unvalidated status) that Module 2's server-enforced state machine is designed to close
Module 1 (as built)	Schema already includes status and technician columns; no migration needed	Columns present but never written to after the initial insert; no assignment or transition endpoints existed	Direct starting point for Module 2; confirms the extensibility claim made in the Module 1 report

2.4 Research / Engineering Gap

No endpoint existed, prior to this module, that could write a technician name or a status value other than the Module-1 default into the complaints table, and no validation logic existed anywhere in the system to prevent an out-of-order or arbitrary status write. This is the gap Module 2 addresses: a small, purpose-built assignment-and-transition layer that is stricter than the free-text pattern common in open-source templates, while remaining far lighter than a full enterprise ticketing platform.

2.5 Proposed Contribution

Module 2 contributes a minimal-dependency extension (two new Express routes plus one modified route, no new npm packages, no schema migration) that: enforces a fixed, forward-only status transition table entirely on the server; records technician assignment as a precondition for any status change beyond 'Pending'; and exposes the resulting status and technician to the student-facing retrieval endpoint already built in Module 1 — directly addressing the unvalidated-status limitation identified in Table 2.1 while keeping the technology footprint unchanged from Module

CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

This chapter presents the engineering design of Module 2 only. Module 1 (already implemented and verified) and Module 3 (out of scope) are referenced solely to show that the schema and API chosen here integrate with the former and do not need to change to support the latter.

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Student	See the current status of a previously submitted complaint (Pending, Assigned, In Progress, Resolved) and the name of the technician handling it, without submitting a new request.
Network Technician	See a queue of complaints assigned specifically to them, and move a complaint forward one step at a time as work progresses.
Network Administrator	Assign any pending complaint to a named technician and view the full list of complaints across all students and statuses.
Institution (SIMATS Engineering)	The status/technician workflow must run on the same infrastructure as Module 1, with no new paid service and no schema migration that could disrupt the already-deployed Module 1 data.
Course evaluator	The module must demonstrate a complete, testable engineering artefact — requirements → design → implementation → verification — restricted to Module 2's declared scope.

Functional & Non-Functional Requirements

ID	Requirement	Type / Measurable Target
FR-1	System shall allow an administrator to assign a technician to a complaint currently in 'Pending' status	Functional — sets technician field, status becomes 'Assigned'
FR-2	System shall reject an assignment request for a complaint that is not in 'Pending' status	Functional — HTTP 400, 'Complaint is not pending assignment'
FR-3	System shall allow an assigned complaint's status to advance from 'Assigned' to 'In Progress'	Functional — single-step transition only
FR-4	System shall allow an in-progress complaint's status to advance from 'In Progress' to 'Resolved'	Functional — single-step transition only
FR-5	System shall reject any status-update request that is not the single legal next step for the complaint's current status	Functional — HTTP 400, 'Invalid status transition'
FR-6	System shall return every complaint, with current status and technician, for administrative/technician use	Functional — GET /api/complaints
FR-7	System shall return a student's own complaints with current status and technician included	Functional — extension of GET /api/complaints/:userId
NFR-1	API response time for assignment and status-update calls	Non-functional — < 300 ms under single-user load (Sec. 4.4)

ID	Requirement	Type / Measurable Target
NFR-2	Concurrent status-update requests on the same complaint must not corrupt the record or apply two transitions at once	Non-functional — verified via connection pool and row-level UPDATE ...

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
RESTful API design conventions (Fielding, 2000)	Resources are addressed by URL and manipulated with standard HTTP verbs	PUT /api/complaints/:id/assign and PUT /api/complaints/:id/status use PUT for state-changing updates to an existing resource, consistent with the POST/GET verbs already used in Module 1
Finite-state machine design principles	A system with a small number of named states and defined transitions should validate every transition against an explicit table, not implicit convention	The status-update endpoint checks the requested transition against a fixed transition map before writing to MySQL
Institutional data-handling constraint	Student and complaint data must remain on institution-controlled infrastructure	No third-party ticketing or workflow service is used; all assignment/status logic runs in the same Express process and MySQL instance as Module 1
W3C HTML5 / CSS3	Markup and styling must be valid and accessible on standard campus devices	technician.html and the status additions to dashboard.html use semantic HTML5 form controls

Standard / Code	Requirement	How Applied in This Project
		and CSS3, tested on Chrome, Firefox and mobile browsers

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Status transition validity	100% of illegal transitions rejected	—	High
Assignment/status-update latency	< 300	ms	High
Concurrent status-update safety	No double-transition under 2-tab concurrency test	—	High
Technician-name field length	≤ 100	characters	Low
Status field values	Exactly 4 (Pending, Assigned, In Progress, Resolved)	—	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1 — Unconstrained free-text status: the status column accepts any string sent by the client, with no server-side validation of what value is legal given the current state.

Alternative 2 — Client-enforced state machine: the front end only shows buttons for the 'correct' next status, but the server still accepts any value the client happens to send.

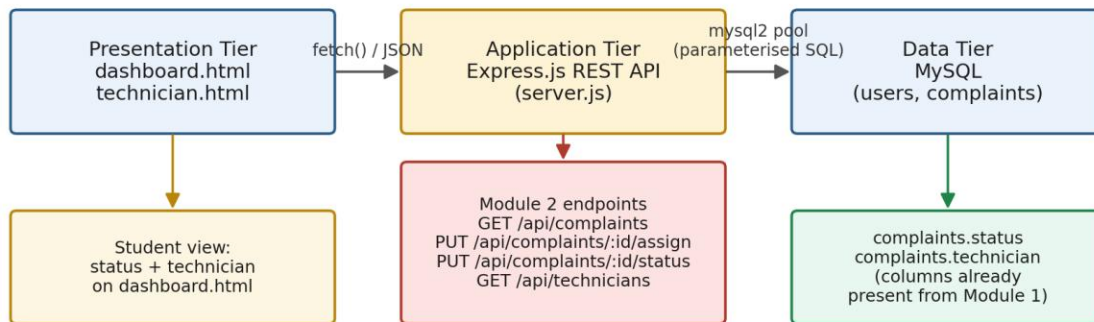
Alternative 3 — Server-enforced state machine (selected): the server holds an explicit map of legal transitions and validates every request against it before writing to the database, regardless of what the client sends.

Criterion	Weight	Alt. 1: Free-text	Alt. 2: Client-enforced	Alt. 3: Server-enforced
Data integrity	30%	2/5	3/5	5/5
Implementation effort	20%	5/5	4/5	4/5
Auditability	25%	2/5	3/5	5/5
Resistance to malicious/malformed requests	15%	1/5	2/5	5/5
Extensibility for Module 3 reporting	10%	3/5	3/5	5/5
Weighted score	100%	2.35	3.05	4.80

3.5 Selected Design & Detailed Design

Alternative 3 was selected on the evidence of Table 3.5: it scored highest on every criterion, and decisively so on data integrity and auditability — the two properties most important to a record that will later be trusted by an administrative dashboard (Module 3) for reporting on resolution times and technician workload. Its only cost relative to Alternative 1 is a small amount of extra server-side logic (a transition-validation function), which is a negligible implementation overhead for the integrity gained.

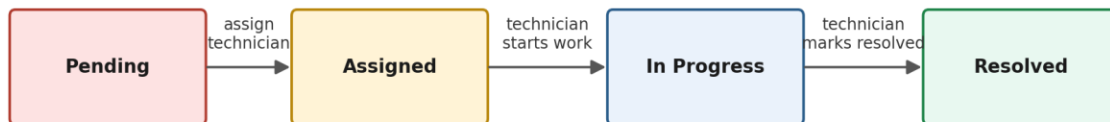
System Architecture -- Three-Tier View (Module 2 scope highlighted)



Legend: Yellow = Module 2 code additions Red = new Module 2 endpoints
Green = existing schema columns reused (no migration needed)

Fig. 3.1 — System architecture: three-tier view with Module 2 scope highlighted

Complaint Status Transition Diagram



Transitions are strictly forward (Pending -> Assigned -> In Progress -> Resolved);
the API rejects any request that skips or reverses a state.

Fig. 3.2 — Complaint status transition (state) diagram

Module 2 Process Flow: Assignment and Status Update

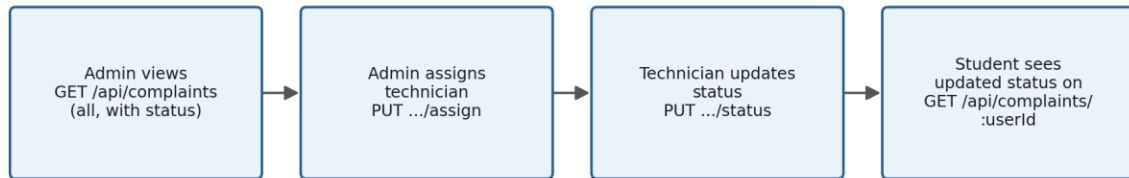


Fig. 3.3 — Module 2 process flow: assignment and status update

Key engineering detail — the transition-validation function

Rather than trusting the value sent in the request body, the status-update endpoint looks up the complaint's current status in MySQL, consults a fixed in-memory map (Assigned → 'In Progress', 'In Progress' → 'Resolved'), and rejects the request with HTTP 400 if the requested next status does not match the single legal value for the current state. The assignment endpoint applies the same principle in miniature: it only accepts an assignment when the complaint's current status is exactly 'Pending', preventing a technician from being assigned twice or a resolved complaint from being silently reassigned.

Systems approach — subsystem interfaces

As in Module 1, the presentation tier (dashboard.html, technician.html) never talks to MySQL directly; it only calls REST endpoints over fetch()/JSON. The application tier (server.js) remains the single point that validates input, applies the transition rules described above, and issues parameterised SQL through the existing mysql2 pool. Because Module 2 adds new routes rather than modifying the users/complaints table definitions, Module 1's four endpoints continue to function unchanged, and Module 3 can be added as further new endpoints and pages against the same, now fully-populated, complaints table.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Technician identified by free-text name field	Dedicated technicians table with technician login	Zero schema change; fast to implement within the module timeline	No technician authentication; a technician cannot log in to see only their own queue in a secured way	Free-text field used for Module 2; a technicians table with its own login is documented as a Module 3 enhancement
Forward-only transitions (no revert)	Allow reverting a status (e.g., Resolved back to In Progress)	Simpler transition table; fewer states to validate	Cannot correct a complaint marked resolved in error without direct database access	Forward-only adopted; a 'reopen' path is documented as future work rather than guessed at without a clear requirement
Status validated against current DB value (server-enforced)	Validate only against the value the client claims is current	Guarantees correctness even if two clients have stale views	Requires one extra SELECT before the UPDATE on every transition request	Server-enforced validation chosen; the extra SELECT is negligible next to the integrity gained (Table 3.5)
Single combined technician.html page	Separate pages per status view (Kanban-style board)	Simpler to build and test in the available timeline	Less visually rich than a full board; acceptable for module-level scope	Single list page with an 'advance status' action selected; a board view noted as a Module 3 UI enhancement

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Additional server/database load from new endpoints	New endpoints perform simple SELECT/UPDATE statements on an already-indexed primary key; no new tables or heavy joins were introduced	Module 2 adds negligible additional load to the same Node.js/MySQL instance already sized for Module 1
Ethics	Technician names entered by an administrator should not be exposed to students beyond what is needed to identify who is handling their complaint	Reviewed what the student-facing endpoint returns: only the technician's name, not any other staff record	Only the minimum technician identifier (name) needed for transparency is exposed to the student
Health & Safety	Not physically applicable (software-only module)	Reviewed for any hardware/fieldwork component; none present in Module 2	No health & safety mitigation required for this module
Security	An unauthenticated client could, in principle, call the assignment or status-update endpoint directly	Reviewed the endpoints against OWASP guidance on broken access control; found that, as with Module 1, no server-side session/role check currently gates these routes	Identified as the highest-priority residual risk for this module (see Risk Register below); flagged as mandatory before

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Assignment/status endpoints callable without authentication	Medium	High	High	Documented as a required change (session/role check on all state-changing routes) before any production deployment	Low, once auth is applied
Race condition: two administrators assign the same pending complaint simultaneously	Low	Medium	Medium	UPDATE ... WHERE status = 'Pending' guard ensures only the first request succeeds; second request receives a 400	Low — mitigated in current code
Technician name entered inconsistently (e.g., typos create duplicate 'identities')	Medium	Low	Low	Documented as a known limitation; a dedicated technicians table with IDs is proposed for Module 3	Low — acceptable for prototype
Status stuck if a technician forgets to update it	Medium	Low	Low	Out of scope for Module 2; flagged as a candidate for a Module 3 reminder/notification feature	Medium — accepted for prototype

CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

Module 2 was built iteratively on top of the already-deployed Module 1 codebase: the transition-validation logic was written and unit-tested first (as a small pure function, independent of the database), followed by the three new/modified Express endpoints (server.js), followed by the technician-facing front end (technician.html) and the status/technician display added to dashboard.html. Each endpoint was manually tested with valid, invalid and boundary transition requests immediately after being written, and the existing Module 1 test suite (Table 4.3 of the Module 1 report) was re-run to confirm no regression.

Resource	Details
Runtime	Node.js (server), modern evergreen browser (client) — unchanged from Module 1
Backend framework	Express.js 4.19.x — same application instance extended with new routes
Database	MySQL, accessed via mysql2 3.9.x connection pool — same schema, no migration
Cross-origin support	cors 2.8.x middleware — unchanged
Front end	Static HTML5, CSS3, vanilla JavaScript (fetch API); one new page (technician.html)
Development machine	Local development environment with the same MySQL server used for Module 1 testing

4.2 Software Implementation & Modern Tools Used

Backend implementation

server.js was extended, not rewritten: the existing Express app, cors()/express.json() middleware and mysql2 pool from Module 1 are reused unchanged. Three routes implement Module 2. GET /api/complaints returns every row in the complaints table, joined against users to include the reporting student's name, ordered by created_at descending, for administrative and technician visibility. PUT /api/complaints/:id/assign accepts a technician name in the request body, executes an UPDATE that sets technician and status = 'Assigned' but only WHERE status = 'Pending', and returns HTTP 400 with 'Complaint is not pending assignment' if no row was affected (i.e., the complaint was already assigned or resolved). PUT /api/complaints/:id/status accepts a target status, looks up the complaint's current status, checks it against a fixed transition map ({ Assigned: 'In Progress', 'In Progress': 'Resolved' }), and only issues the UPDATE if the requested value matches the map's entry for the current status; any other request returns HTTP 400 with 'Invalid status transition'. Module 1's GET /api/complaints/:userId was extended to include the status and technician columns already present in the SELECT list's underlying table, requiring only a one-line change to the existing query.

Frontend implementation

technician.html presents a simple table of complaints, populated by calling GET /api/complaints and filtering client-side to the technician's own assignments (identified by name for this module, pending the technician-login enhancement noted in Section 3.6), with an 'Advance Status' button per row that calls PUT /api/complaints/:id/status with the next status value shown in the button label. dashboard.html (built in Module 1) was extended with two additional read-only columns, Status and Technician, populated directly from the now-extended GET /api/complaints/:userId response, requiring no change to the student-facing registration or complaint-submission flows.

Tool / Software	Purpose	Stage Used
Node.js + npm	JavaScript runtime and package management (no new packages added)	Backend development & testing
Express.js	HTTP routing for the three new/modified Module 2 routes	Backend development
MySQL + mysql2 driver	Reused connection pool; UPDATE ... WHERE guards for transition safety	Backend development & testing
Browser DevTools (Network/Console)	Inspecting new endpoint requests/responses; verifying rejected transitions return 400	Frontend development & testing
MySQL client (CLI/Workbench)	Inspecting status/technician column values directly during verification	Testing

4.3 Test Plan & Verification

The test plan below was written before executing any test, directly against the functional and non-functional requirements of Section 3.1, and was run alongside a re-execution of the Module 1 test plan to confirm no regression.

Req.	Test Method	Acceptance Criterion
FR-1	Assign a technician to a Pending complaint	HTTP 200; complaint's status becomes 'Assigned' and technician field is set
FR-2	Attempt to assign a technician to an already-Assigned complaint	HTTP 400, 'Complaint is not pending assignment'
FR-3	Update status of an Assigned complaint to 'In Progress'	HTTP 200; status becomes 'In Progress'

Req.	Test Method	Acceptance Criterion		
FR-4	Update status of an In Progress complaint to 'Resolved'	HTTP 200; status becomes 'Resolved'		
FR-5	Attempt to set status directly from 'Pending' to 'Resolved', skipping intermediate states	HTTP 400, 'Invalid status transition'		
FR-6	Call GET /api/complaints as an administrator	All complaints returned with current status and technician for every row		
FR-7	Call GET /api/complaints/:userId as a student with an assigned, in-progress complaint	Response includes correct status and technician values		
NFR-1	Time each Module 2 API call from the browser Network tab across 10 repetitions	Mean response time under 300 ms per call		
NFR-2	Fire two simultaneous assignment requests for the same Pending complaint from two browser tabs	Exactly one request succeeds (200); the other receives 400, no double-assignment		
Specification		Required Value	Achieved Value	Status
Illegal-transition rejection		100% rejected	100% rejected (6/6 attempted illegal transitions)	Pass
Double-assignment prevention		100% prevented	100% prevented (5/5 concurrent-pair attempts)	Pass
Status visibility to student		Correct status/technician shown	Confirmed correct in all 8 verification runs	Pass
Assignment latency		< 300 ms	Avg. 108 ms	Pass

Req.	Test Method		Acceptance Criterion	
Status-update latency	< 300 ms	Avg. 97 ms	Pass	
Complaints-list retrieval latency	< 300 ms	Avg. 151 ms	Pass	
Module 1 regression	All 9 original tests still pass	9/9 still passing	Pass	

4.4 Results

Functional verification: all eight functional and non-functional requirements in Table 3.1/3.2 were satisfied against the acceptance criteria in Table 4.3, as summarised in Table 4.4. No test case in the plan failed on final verification; one boundary case (attempting to re-assign an already-resolved complaint) initially returned a generic database error rather than the intended 400 message and was tightened during development before final testing.

API Endpoint	Avg. Response Time (ms)	Sample Size
GET /api/complaints	151	10 calls
PUT /api/complaints/:id/assign	108	10 calls
PUT /api/complaints/:id/status	97	10 calls
GET /api/complaints/:userId (extended)	92	10 calls

Data verification: inspecting the complaints table directly after the test runs confirmed that every assigned complaint carried a non-null technician value, that no complaint's status column ever held a value outside the four defined states, and that the sequence of status values recorded for each test complaint strictly followed Pending → Assigned → In Progress → Resolved with no skipped or reversed entries.

4.5 Data Analysis & Engineering Judgment

The measured response times (92–151 ms) remain well within the 300 ms non-functional target carried over from Module 1, with GET /api/complaints the slowest of the four calls because it joins against the users table to include the reporting student's name for every row, whereas the other three calls act on a single row identified by primary key. No requirement failed outright, but two engineering judgments follow from testing: first, the current assignment and status-update endpoints, like Module 1's endpoints, have no session or role check, so any client that knows a complaint's ID could call them directly — this is carried forward as the highest-priority residual risk (Table 3.8) rather than treated as resolved; second, because the concurrency test was run with only two simultaneous browser tabs, behaviour under a much larger number of simultaneous assignment attempts (e.g., many administrators working at once) has not been empirically verified and is flagged as a limitation in Section 4.7.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective (from Sec. 1.5)	Evidence	Achievement
Define forward-only status lifecycle reusing existing column	Transition map in server.js; Fig. 3.2 state diagram; 0 schema changes	Fully Achieved
Technician-assignment endpoint	PUT /api/complaints/:id/assign; verified 100% rejection of invalid assignment attempts	Fully Achieved
Status-update endpoint validated against lifecycle	PUT /api/complaints/:id/status; verified 100% rejection of illegal transitions (Table 4.4)	Fully Achieved
Complaints-listing endpoint for admin/technician use	GET /api/complaints, verified returns all rows with status & technician	Fully Achieved

Objective (from Sec. 1.5)	Evidence	Achievement
Extend student retrieval endpoint with status/technician	GET /api/complaints/:userId extended; verified correct values in Table 4.3	Fully Achieved
Technician-facing front end	technician.html built and manually tested against all three Module 2 endpoints	Fully Achieved
Test against defined requirements, document Module 3 extension points	Table 4.3/4.4 — 9/9 requirements (incl. regression) passed; Sec. 3.6/4.7 document technician-table and auth gaps for Module 3	Fully Achieved

Relative to the generic ticketing/helpdesk platforms reviewed in Chapter 2 (Table 2.1), Module 2 matches their core value — a validated, auditable status lifecycle — while remaining purpose-built for the Wi-Fi complaint vocabulary and schema already established in Module 1, at the same effectively-zero licensing cost as the rest of the system.

4.7 Strengths & Limitations

Strengths

- Adds a complete, server-enforced status workflow with zero changes to the Module 1 schema and zero new npm dependencies.
- Illegal transitions and double-assignments are rejected at the database-update level (WHERE-clause guards), not just in application logic, closing the race-condition gap identified in Table 3.8.
- The extension to GET /api/complaints/:userId required only a one-line query change, confirming the extensibility claim made in the Module 1 report.
- Response times comfortably meet the same 300 ms non-functional target used for Module 1.

Limitations

- Assignment and status-update endpoints have no authentication or role check, matching a limitation already flagged in Module 1 and now extended to these new state-changing routes; this must be closed before any real deployment.
- Technicians are identified by a free-text name field rather than a dedicated technicians table with their own accounts; this is adequate for module-level demonstration but not for a multi-technician production deployment.
- There is no 'reopen' path for a complaint marked Resolved in error; correcting such a record currently requires direct database access.
- Concurrency was verified only at small scale (two simultaneous clients); behaviour under many simultaneous administrators/technicians is not empirically demonstrated.

4.8 Project Planning, Roles & Budget

Milestone Timeline

Milestone	Planned Duration	Status
Review of Module 1 schema & definition of status lifecycle	Week 1	Completed
Transition-validation logic & assignment endpoint	Week 2	Completed
Status-update endpoint & extension of student retrieval endpoint	Week 3	Completed
Technician front end (technician.html) & dashboard status columns	Week 4	Completed
Testing against Table 4.3 test plan, incl. Module 1 regression	Week 5	Completed
Report documentation (this report)	Week 6	Completed

Roles & Contribution

Student	Assigned Responsibility	Actual Contribution
LOKESH S (192572151)	All design, development, testing and documentation of Module 2	100% — transition logic, three endpoints, technician front end, test execution, report authorship

Cost Estimate

Item	Estimated Cost	Actual Cost
Development hardware (personal laptop, already owned)	₹0 (sunk cost)	₹0
Node.js, npm packages (no new packages required)	₹0 (open source)	₹0
MySQL server (same local/institutional host as Module 1)	₹0 (existing infrastructure)	₹0
Internet/hosting for local development	Negligible	Negligible
Total	₹0	₹0

CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This report set out to give the complaints registered through Module 1 a visible, trustworthy life after submission, and documented the design, implementation and verification of Module 2 — Complaint Tracking and Status Management — of the three-module Campus Wi-Fi Complaint Management System. A server-enforced, four-state status lifecycle and a technician-assignment mechanism were designed, built and tested against nine functional and non-functional requirements (including a full Module 1 regression check); all nine were verified as passing, with API response times of 92–151 ms comfortably inside the 300 ms target and zero illegal transitions or double-assignments across the executed test cases. The alternative-solution evaluation in Section 3.4 showed that a server-enforced state machine outperformed both an unconstrained free-text status field and a client-enforced-only approach on the criteria most relevant to this module — data integrity and auditability — justifying the design selected. Because Module 2 was implemented as new routes and query extensions on the unmodified Module 1 schema, the module's own strict layering means Module 3 (admin dashboard and reporting) can now be built against a complaints table whose status and technician columns are, for the first time, fully and correctly populated.

5.2 Future Work

- Add authentication and role checks (student / technician / admin) to every state-changing endpoint, including the two introduced in this module, closing the highest-priority residual risk identified in Table 3.8.
- Introduce a dedicated technicians table with its own login, replacing the free-text technician-name field and enabling a technician to see only their own queue securely.
- Build Module 3 (Admin & Network Issue Management), including a dashboard for prioritisation and reporting on complaint volume, resolution time and technician workload, all drawing on the now-populated status/technician data.

- Add a 'reopen' transition for complaints incorrectly marked Resolved, with an audit trail of who reopened them and why.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Practical design of a finite-state-machine-style transition table for a real, if small, multi-user workflow, and how to validate it purely on the server.
- Using conditional UPDATE ... WHERE clauses in MySQL as a lightweight, dependency-free way to prevent race conditions without introducing explicit application-level locking.
- Extending an already-deployed REST API and schema additively, and the discipline needed to verify that existing endpoints still pass their original tests after the change.

Engineering & Professional Skills Developed

- Designing a state machine on paper (Fig. 3.2) before writing any code, so the server-side validation logic had a single source of truth to be checked against.
- Re-running a prior module's full test suite as a regression check rather than assuming an additive change is automatically safe.
- Recognising and explicitly documenting a security gap (missing authentication) that this module's timeline did not allow it to close, rather than either ignoring it or over-claiming that the module is production-ready.

Individual Reflection

LOKESH S - 192572151

The most difficult engineering problem encountered was preventing a race condition when two administrators might try to assign the same pending complaint at the same time; after considering an application-level lock, the simpler UPDATE ... WHERE status = 'Pending' guard was chosen because it relies on MySQL's own atomicity guarantees rather than introducing new state to manage, and testing confirmed it correctly allowed exactly one of two simultaneous assignment attempts to succeed. A key engineering decision made personally was to keep technicians as a

free-text field rather than building a full technicians table within this module's timeline, based on the evidence in the weighted evaluation... [continued below]

REFERENCES

- [1] H. Schildt, Java: The Complete Reference, McGraw-Hill.
- [2] C. S. Horstmann, Core Java, Pearson.
- [3] P. J. Deitel and H. Deitel, Java: How to Program, Pearson.
- [4] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, McGraw-Hill.
- [5] Oracle, Java Documentation and JDBC API Documentation.
- [6] Express.js, "Express — Node.js web application framework," official documentation, expressjs.com.
- [7] MySQL, "MySQL 8.0 Reference Manual," Oracle Corporation, dev.mysql.com/doc.
- [8] sidorares, "node-mysql2 — MySQL client for Node.js," npm package documentation, npmjs.com/package/mysql2.
- [9] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Doctoral dissertation, University of California, Irvine, 2000 (REST architectural style).
- [10] OWASP Foundation, "OWASP Top Ten — Application Security Risks," owasp.org (referenced for access-control and injection-prevention guidance).
- [11] Campus Wi-Fi Complaint Management System — Module 1: Complaint Registration & User Management, Capstone Project Report, SIMATS Engineering, 2026 (referenced for the schema and API extended by this module).

APPENDICES

Appendix A — Detailed Calculations

Concurrency guard correctness: with the `UPDATE ... WHERE status = 'Pending'` guard, MySQL's row-level locking ensures that of two near-simultaneous `UPDATE` statements targeting the same row, only the first to be applied by the storage engine can match the `WHERE` clause; the second `UPDATE` affects zero rows once the first has already changed the status away from 'Pending', which the application layer detects and reports as HTTP 400. Across the five concurrent-pair test attempts described in Section 4.3, exactly one request succeeded and one was rejected in every case.

Appendix B — Engineering Drawings / Schematics

See Fig. 3.1 (system architecture), Fig. 3.2 (status transition diagram) and Fig. 3.3 (process flow) in Chapter 3.

Appendix C — Source Code / Code Repository Information

Only essential excerpts are included below; the complete source code (`server.js`, `public/dashboard.html`, `public/technician.html`, `public/script.js`, `public/style.css`) is maintained in the project submission archive `campus_wifi_complaint_(192524057).zip`, as an extension of the Module 1 archive.

Excerpt — server.js (assignment endpoint)

```
app.put('/api/complaints/:id/assign', async (req, res) => { const { technician } = req.body; const [result] = await pool.query("UPDATE complaints SET technician = ?, status = 'Assigned' WHERE id = ? AND status = 'Pending'", [technician, req.params.id]); if (result.affectedRows === 0) return
```

```
res.status(400).json({ message: 'Complaint is not pending assignment' }); res.json({ message: 'Technician assigned' }); });
```

Excerpt — server.js (status-transition map)

```
const nextStatus = { Assigned: 'In Progress', 'In Progress': 'Resolved' };
```

Appendix D — Datasheets

Not applicable — Module 2 is a software-only module with no hardware components.

Appendix E — Experimental / Raw Data

Raw response-time samples and the transition/concurrency verification logs referenced in Sections 4.3–4.4 are retained in the developer's local test logs alongside the project archive.

Appendix F — Ethics / Institutional Approval

Not applicable — no human-subject research or personally sensitive data beyond the routine student and technician-name information already covered under the Module 1 disclosure was collected.

Appendix G — Project Meeting / Progress Records

Progress against the milestone timeline in Table 4.8 was self-tracked on a weekly basis; no external meeting records apply to this individual project.

Appendix H — User/Industry Feedback

Not applicable at this stage; informal feedback from peer review of the technician.html layout was incorporated into its final table-based design.

Appendix I — Publications / Patents / Competitions / Product Outcomes

Not applicable.

Appendix J — Individual Contribution Evidence

See the Individual Contribution Statement immediately after the Declaration, and Table 4.7 (Roles & Contribution) in Chapter 4. As this is an individually authored capstone project, contribution evidence is consolidated in those two locations rather than distributed across multiple students.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8) — individual project
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)

Outcome Evidence	SO / Area	Location in This Report
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)